QCalc design documents (v1.3.0)

Nov 13, 2026 · @津嘉山 弘

This is the overview of the design documents of **QCalc 1.3.0** (versionCode 5, package name `io.github.hiroshitsu.qcalc`): its requirements, design, use, maintenance and tests, written to match the current source code. Every document is in English and in Japanese, with the same content: the English tabs are this overview and tabs 1 to 6 in English, and the Japanese tabs are 概要 and tabs 1 to 6 in Japanese; section 11 lists them all. References such as “detailed design 3.6” point to the section where a topic is described in detail.

1. What QCalc is

QCalc is a multi-function calculator for Android. It runs **libqalculate 5.13.2**, the calculation engine of the open-source desktop calculator Qalculate!, without modifying it, and adds a user interface for phones. The user interface is in English (the default) and Japanese.

- **Why**: Qalculate! has versions for PCs (GTK, Qt and the command-line `qalc`) but no official Android version. The goal is to make all of its calculation features usable on an arm64 Android phone and to publish the app on Google Play. The reference device is a SHARP AQUOS sense8 (Android 16, 1080 × 2432).
- **What users do**: type expressions such as `60 mph to km/h`, `100 USD to JPY`, `solve(x^2 - 5x + 6 = 0)` or `integrate(x^2 sin(x), 0, pi)`. The syntax and the results are the same as those of `qalc`.
- **What QCalc adds to libqalculate**: only ten traditional Japanese units (坪, 反, 寸, 尺, 間, 里, 合, 升, 匁, 貫) and the matrix trace `trace()`, both added to the running engine at start-up.
- **What 1.3.0 changed**: the screens are in English and Japanese, with English as the default (“Language / 言語” in the menu or in Settings switches them). QCalc Pro, a one-time purchase in Google Play, unlocks precision above 1,000 digits, 3D graphs, seven color themes and exporting the history as a CSV file; everything else stays free. Builds without the Billing Library, such as the 1.3.0 App Bundle made by the shell scripts, lock nothing, and users from before Pro keep the Pro features for free. The live preview now shows all the digits like the history (a very long result shows its beginning and its end), and digit grouping puts commas only in the integer part. The table “Changes in 1.3.0” below lists the changes.
- **What 1.2.0 changed**: the precision limit was raised from 1,000 to 200,000 digits, the most that works well on Android (the clipboard takes about 1 MB at a time). Long results are shown, copied and kept in full: the history keeps the newest results whole

up to 1,000,000 characters in total. ans is given back with all its digits after a restart (1.1.0 rounded it to about 38 digits), and long numbers in it are read directly with GMP. A result too long to copy gives a message instead of a crash.

- **What 1.1.0 changed**: four keypads switched with tabs (123, functions, calculus and matrices), matrix results shown as tables, built-in examples (12 topics, 69 expressions), undo for a cleared expression or history, three text sizes, Japanese names and search for functions, units and constants, and a workaround for a libqalculate 5.13.2 problem that hid later warnings (section 5).

In scope	Out of scope
Everything libqalculate calculates: arithmetic and functions, units, currencies, equations (`solve`), derivatives and integrals, matrices, number bases, dates, uncertainty propagation, complex numbers, data sets, up to 200,000 digits (above 1,000 with QCalc Pro)	Devices other than arm64 (32-bit ARM, x86)
Input with the on-screen keys and the phone keyboard, live preview, history, `ans` , cancelling a calculation	gnuplot plotting and the file and command functions such as `command()` , `load()` and `export()` (QCalc draws its own graphs)
Lists of functions, variables and units with search, 2D and 3D graphs, user-defined variables and functions, exchange-rate updates, settings, examples, color themes, CSV export of the history	RPN mode, user interfaces in languages other than English and Japanese, sync between devices
Distribution on Google Play, QCalc Pro as a one-time purchase, license texts in the app, published source code	Eigenvalues as a single function, differential equations above first order, values of infinite sums (libqalculate cannot do these)

Changes in 1.3.0. The page (`app.js` , `style.css` , `index.html`), the Java code (`MainActivity.java` , the new `Pro.java` , and `PlayBilling.java` , which only builds with billing contain) and a part of the bridge program `qalc-engine.cc` (`tail_chars` , which returns the end of a long result) changed. libqalculate itself did not.

Change	What	Details in
Language of the screen	English and Japanese. English is the default, and a language the user chooses is saved in `qalc.lang`. The engine and the Java code report in English; on the Japanese screen the page translates their messages (57 kinds)	Requirements definition 1.5 (F-30); detailed design 3.14 and 3.21
QCalc Pro	A one-time product in Google Play (`qcalc_pro`). Precision above 1,000 digits, 3D graphs, color themes and exporting the history are Pro features. Users of builds without billing get them for free (legacy)	Requirements definition 1.9; detailed design 3.20
Color themes	Seven colors for the keys and highlights, in both the light and the dark theme	Detailed design 3.22
Exporting the history	A CSV file (UTF-8 with a byte order mark), protected so that spreadsheets never run a cell as a formula	Detailed design 3.23
All digits in the preview	While typing, results of up to 450,000 bytes are shown; a longer one keeps its beginning and its end (`tail_chars`)	Detailed design 3.5, 3.8 and 3.9
Digit grouping	Commas only in the integer part; the decimals and the space before a unit keep narrow spaces	Detailed design 3.9
Version	versionCode 5 (4 was used by an App Bundle built before English became the default)	Build and maintenance manual 5.7

Change	What	Details in
Tests	27 new page tests (Q01 to Q27). The comparison with qalc (309 expressions), the engine tests, the examples of both screens, the 31 Java tests, the tests on the Japanese screen and the 22 package checks were run again. 665 items in all, all passed	Test specification and results

2. Requirements in brief

Requirements definition 1.5 lists 34 functional requirements (F-01 to F-34; F-22 to F-28 were added in 1.1.0, F-29, long results, in 1.2.0, and F-30 to F-34 in 1.3.0). F-01 to F-33 are met in 1.3.0; F-34, buying QCalc Pro, works only in builds with the Google Play Billing Library (requirements definition 1.9).

- **Calculation** (F-01): the full Qalculate! feature set with the same syntax. libqalculate 5.13.2 is used unmodified, and the typed expression is passed on as it is.
- **Input** (F-02, F-03, F-14, F-22 to F-24): four 5 × 5 keypads plus eight keys that are always shown, the phone keyboard, and automatic conversion of full-width characters. Calculus keys insert templates such as `integrate(, 0, 1)` with the cursor where the expression goes, and an a=b key inserts the “=” of equations.
- **Results** (F-04 to F-08, F-15): a live preview 160 ms after the last key (cut off after 1.5 s), confirming with “=” (limited to 20 s), a history of up to 300 items, `ans`, reuse and copy of results, and cancelling. Inserting a result uses the exact value returned by the engine.
- **Lists and definitions** (F-09, F-11): lists of functions, variables and units from the engine, which on the Japanese screen can also be searched by Japanese names, and user-defined variables and functions saved in the engine's definition files.
- **Graphs** (F-10, F-19): 2D graphs of up to 6 functions, and 3D graphs of up to 4 surfaces on a 25 × 25, 41 × 41 or 61 × 61 grid, computed by the engine and drawn by the page.
- **Data** (F-12, F-18): exchange rates from the same three sources as Qalculate!, updated by hand or automatically after 7 days. The history, the settings, the draft and `ans` are written to files at once.
- **Usability** (F-13, F-16, F-17, F-25 to F-27, F-31): 19 settings, dark mode and seven color themes, a landscape layout, built-in examples to tap and try, undo, and three text sizes (1, 1.18 and 1.38 times).

- **Distribution** (F-20, F-21): a privacy policy on GitHub Pages, the full license texts of the app and its seven bundled components, and an App Bundle with Play App Signing.
- **Additions** (F-28): the Japanese units and `trace()`, added by the bridge program at start-up and never written to the user's definition files.
- **Long results** (F-29, F-33): results of up to 200,000 digits are shown, copied and kept with all their digits in the history and in ans and, since 1.3.0, also in the live preview. A result over 450,000 bytes shows its beginning and its last 20,000 bytes.
- **Language** (F-30): English by default and Japanese by choice ("Language / 言語" in the menu or in Settings). The page holds both languages; the Java code and the engine report in English, and on the Japanese screen the page translates their common messages (57 kinds).
- **Export** (F-32): the history as a CSV file (UTF-8 with a byte order mark), saved where the user chooses with Android's file picker, without a storage permission.
- **QCalc Pro** (F-34): a one-time purchase in Google Play that unlocks precision above 1,000 digits, 3D graphs, color themes and the CSV export (requirements definition 1.9). Builds without the Billing Library lock nothing, and users from before Pro keep the Pro features for free.

The non-functional requirements (requirements definition 1.6) come down to one idea: whatever is typed, the app must never freeze, and the value shown, the value saved as `ans` and the value inserted from the history must always be the same.

- **Performance**: the preview is cut off after 1.5 s, a confirmed calculation after 20 s, a 2D graph after 12 s and a 3D graph after 15 s. Every request carries its own time limit, and the engine starts together with the page.
- **Reliability**: the app never freezes and recovers from engine crashes, thanks to three layers of time limits (section 6). A request that crashed the engine is never sent again; a request is resent only when it could not be written.
- **Accuracy**: each expression is evaluated once, the results match `qalc` (309 expressions compared), and warnings are never lost, because message stops are closed before every command.
- **Data safety**: files are written to a temporary file, synced with fsync and then renamed. Unreadable definition files are moved aside, never overwritten.
- **Security and privacy**: the WebView shows only the bundled page, `command()` is disabled at build time, and the only network traffic is downloading exchange rates (and, in builds with billing, talking to Google Play). No personal data is collected.
- **Compatibility**: Android 7.0 to 16 with targetSdk 36 (edge-to-edge display on Android 15+, back handling on Android 13+), and a static arm64 executable aligned for 16 KB pages.

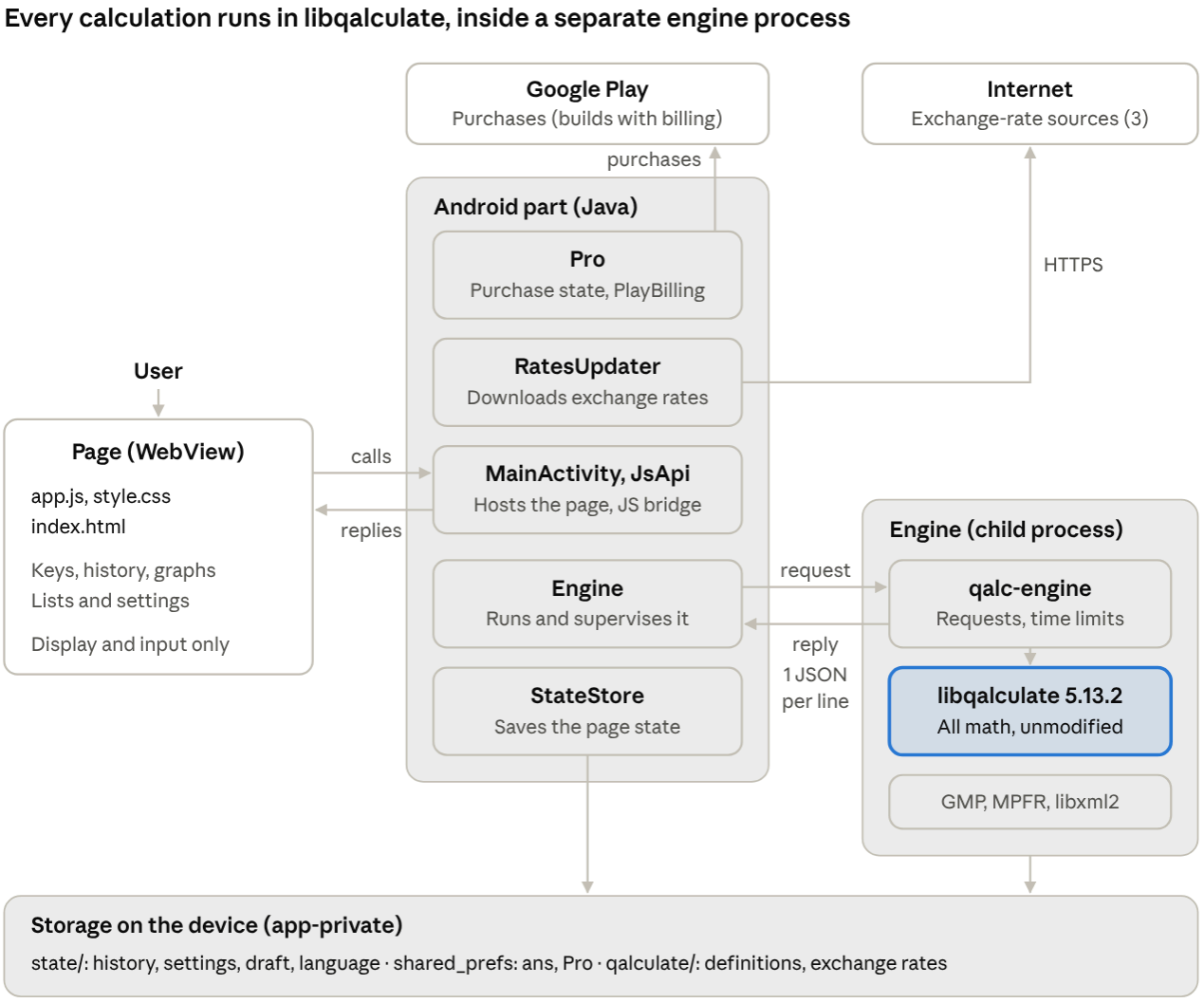
- **Maintainability and license**: libqalculate is unmodified, all builds are reproducible with four shell scripts, the app is GPL v2 or later (with a permission to link the Google Play Billing Library), and the sources of every bundled library are published.

Main constraints (requirements definition 1.7):

1. The source of libqalculate is not changed. Everything specific to QCalc lives in a small bridge program that calls libqalculate's public API.
2. The engine is built with zig as a cross compiler, without the Android NDK. The APK and the App Bundle are assembled with Ubuntu's Android tools (aapt2, dx, zipalign, apksigner, jarsigner) and android.jar from the Android SDK. The build with billing for QCalc Pro is made with Gradle instead (qcalc.billing=true), because the Billing Library comes from Google's Maven repository, which the build environment cannot reach.
3. The engine is a Linux executable stored as `lib/arm64-v8a/libqalcengine.so`. Android 10 and later do not let an app run files from its data directory, so the engine is started from the native library directory, where Android extracts it at install time.
4. targetSdk is 36, as Google Play requires for new apps, so the app supports edge-to-edge display (Android 15+) and the new back handling (Android 16).
5. Google Play signs the app (Play App Signing). The developer keeps only an upload key, which is never published.
6. The name and the icon differ from those of Qalculate!, and the store listing says that QCalc is not an official Qalculate! app.
7. libqalculate has no Japanese translation, so the engine's messages and list names are in English. On the Japanese screen, the page translates the 57 most common messages of the engine and the Java code and the category names, and shows Japanese names beside functions, units and constants.
8. English is the default at every start until a language is chosen, also on phones set to Japanese and for users updating from 1.2.0 (requirements definition 1.7).

Limits of libqalculate itself are explained to users instead of being worked around: `limit()` always warns that its result may be wrong, `dsolve()` solves only first-order equations, infinite sums are not evaluated, there is no eigenvalue function, and one-letter names such as `a` or `m` may be read as units.

3. Architecture



Architecture · the page, Java, the engine process and storage

The user only touches the page in the WebView. Every calculation runs in libqalculate inside a separate engine process. The Java layer connects the two, supervises the engine, saves data, downloads exchange rates and keeps the state of QCalc Pro (basic design 2.1–2.3).

The parts meet only through three fixed interfaces, and each of them passes plain strings (JSON). This is what makes it possible to test the page, the Java code and the engine separately (section 9).

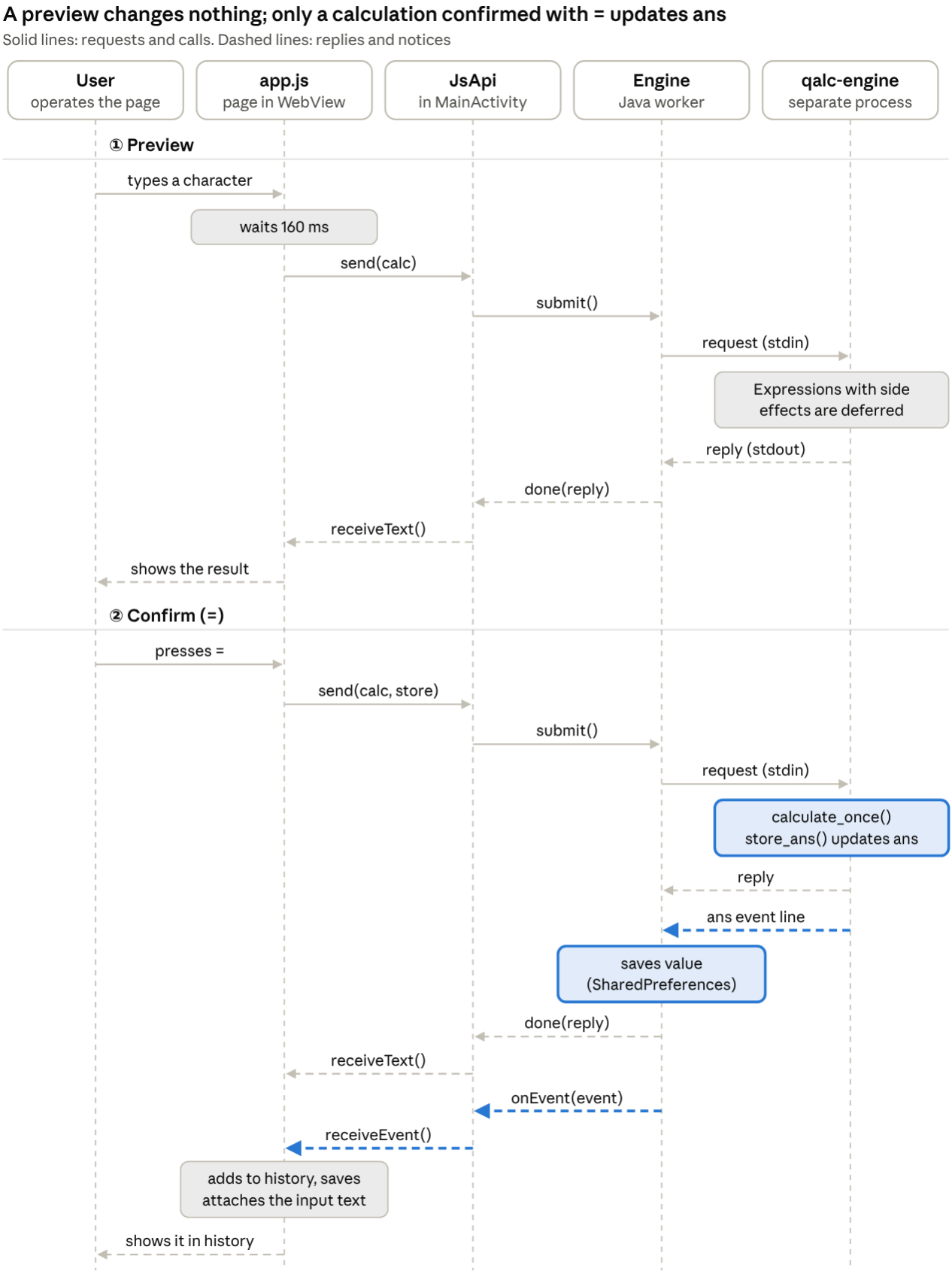
Interface	Direction	Contents
`QalcNative`	Page → Java	Fourteen methods that `JsApi` exposes to the WebView: `send`, `cancel`, `updateRates`, `loadState`, `saveState`, `copy`, `appInfo`, `openUrl`, and since 1.3.0 `proInfo`, `buyPro`, `restorePro`, `refreshPro`, `setTestPro` and `saveFile`
`QalcBridge`	Java → page	Callbacks provided by `app.js`: `receiveText` for replies and `receiveEvent` for events (`ans`, and since 1.3.0 `pro` when the state of QCalc Pro changes and `saved` with the result of saving a file), called with `evaluateJavascript`
JSON lines	`Engine` ⇄ `qalc-engine`	One request per line on the engine's standard input; one reply or event per line on its standard output

File	Lines	Role
`app/assets/index.html`	51	Page layout: top bar, history, input field, keypad and menu containers
`app/assets/style.css`	744	Page look: light and dark colors, seven color themes, keys and tabs, matrix tables, text sizes, panels, graphs, landscape layout
`app/assets/app.js`	2,653	Page logic: English and Japanese texts, keys and templates, input, preview, confirm, history, matrix tables, undo, examples, lists and search, 2D and 3D graphs, definitions, rates, settings, the QCalc Pro panel, CSV export, licenses
`MainActivity.java`	574	`MainActivity` and `JsApi`: the WebView and its settings, the JavaScript bridge, window insets (Android 15+), back handling, lifecycle, setting up QCalc Pro, saving files
`Engine.java`	357	Starts, supervises and restarts the engine; matches replies to requests; saves and restores `ans`
`RatesUpdater.java`	164	Downloads, checks and replaces the exchange-rate files

File	Lines	Role
`StateStore.java`	123	Writes the page state (history, settings, draft) to files at once
`Pro.java`	213	The state of QCalc Pro (owned, pending, earlier users, price) and the kind of billing (play, debug or none); loads PlayBilling by its class name, so that builds without it still work
`PlayBilling.java`	296	Purchases with the Google Play Billing Library 8: the connection, the price, the purchase screen, acknowledging. Only in builds with billing (kept in app/src-billing)
`bridge/qalc-engine.cc`	1,643	The engine's bridge: JSON protocol, calls into libqalculate, the beginning and the end of long results, graph values, `ans`, watchdog, protection of definition files, Japanese units and `trace()`, closing message stops
libqalculate, GMP, MPFR, libxml2	—	The engine's core: official sources (5.13.2, 6.3.0, 4.2.2 and 2.15.4), linked statically

Deployment (basic design 2.2). Google Play receives the App Bundle signed with the upload key, builds an APK for each device (about 3.3 MB for arm64) and signs it with the app signing key that Google keeps. When the app is installed, Android extracts `libqalcengine.so` into the native library directory (`android:extractNativeLibs="true"`). Despite its name, it is an executable: `Engine` starts it with `ProcessBuilder` as a child process in the working directory `files/`, with these environment variables: `HOME`; `QALCULATE_USER_DIR` (`files/qalculate/`, where libqalculate keeps user definitions and exchange rates); `TZ` (the current UTC offset in POSIX form, because a static Linux program cannot read Android's time-zone data); `LANG=C.UTF-8`; and `QALC_CURRENCY` (the currency of the phone's region, JPY in Japan).

4. Main flow: preview and confirm



Sequence diagram · live preview and confirmed calculation

The live preview and the confirmed calculation are both `calc` requests (detailed design 3.5). They differ only in four fields: `store` (whether `ans` is updated), `preview` (whether expressions with side effects are skipped), `timeout` and the maximum length of the result.

- **Preview**: 160 ms after the last key, the page sends `calc` with a 1.5 s limit. Expressions that would change something — assignments such as `x = 5`, `save()`, or user functions that save — are not run in a preview. The engine answers `deferred` instead, so typing never defines or saves anything by accident.
- **Confirm (=)**: `calc` with `store:true` and a 20 s limit. The engine evaluates the expression exactly once (`calculate_once`: the same steps as `calculateAndPrint` in `qalc`, but with a single evaluation), so random values or expressions like `a = a + 1` are never evaluated twice, and the value shown is the value stored.
- **ans**: after a confirmed calculation, the engine shifts `ans5 ← … ← ans` as `qalc` does and sends an `ans` event line after the reply. Java saves its `value` in SharedPreferences and gives it back with `set_ans` whenever a new engine process starts, so `ans` survives restarts. The event's `input` field is the exact text that the history inserts when the user taps a result (for example `7/3`, not the rounded `2.333333333`).

5. The engine protocol

The app writes one JSON request per line to the engine's standard input, and the engine answers with one JSON line on its standard output, with the same `id` (detailed design 3.3). Requests are handled one at a time.

- Success: `{"id":…, "ok":true, …}`. Failure: `{"id":…, "ok":false, "error":"…"}`.
- `"restart":true` in a reply means that the engine exits after the reply; Java starts a new process for the next request.
- Anything that libqalculate prints by itself is moved to standard error at start-up, so it never mixes with the replies.
- Error texts and messages are in English. Before 1.3.0 the bridge program's own texts were in Japanese; now the page translates them on the Japanese screen (detailed design 3.14 and 3.21).

Command	Main request fields	Main reply fields
`init`	—	Engine version, time of the exchange rates, whether `ans` is defined

Command	Main request fields	Main reply fields
`calc`	`expr`, `opts` (settings), `timeout` (100–120,000 ms), `store`, `preview`, `max_chars`, and since 1.3.0 `tail_chars` (how much of the end of a long result to return)	`result`, `parsed`, `approx`, `comparison`, `timeout`, `truncated`, `chars` and `digits` (when cut), `tail` (since 1.3.0), `messages`, `deferred`
`set_ans`	`expr`	`set`
`plot`	`exprs` (up to 12), `min`, `max`, `steps`, `var`, `timeout`	`series` (x and y values for each expression), `messages`
`plot3d`	`exprs` (up to 4), `xmin`, `xmax`, `ymin`, `ymax`, `nx` and `ny` (2–121), `timeout`	`x`, `y`, `series` (z values on the grid, `complete`), `messages`
`catalog`	`kind` (functions, variables or units)	`items`: names, titles, categories, descriptions and the text to insert
`rates_info`, `rates_reload`	—	Sources and file names; whether new rates were loaded
`define_var`, `define_func`, `delete_def`, `user_defs`	A name and a value or an expression	`saved`; the list of user definitions

```
{"id":19,"cmd":"calc","expr":"7/3","opts":
{"approx":"auto","precision":10},"timeout":20000,"store":true,"max_chars":450
000,"tail_chars":20000}
{"id":19,"ok":true,"result":"2 + 1/3 ≈ 2.333333333","parsed":"7 /
3","approx":false,"comparison":false,"timeout":false,"truncated":false,"messa
ges":[]}
{"event":"ans","id":19,"value":"7/3","input":"7/3"}
```

A workaround for libqalculate 5.13.2 (detailed design 3.19). After a definite integral with an infinite bound, libqalculate leaves one level of “message stop” (`beginTemporaryStopMessages`) open, and from then on no warning or error is shown; the official `qalc` behaves the same way. The bridge program closes any open message stop before every command, so QCalc keeps showing warnings such as division by zero. libqalculate itself is not changed.

6. Time limits and recovery

Time limits work in three layers (detailed design 3.13). The innermost one normally ends a request; the outer ones are safety nets.

Request	Engine limit (`timeout`)	Engine watchdog (+3 s)	Java wait (timeout × expressions + 10 s)	Page's last safety net
Preview	1.5 s	4.5 s	11.5 s	120 s
Confirm (=)	20 s	23 s	30 s	120 s
2D graph (6 expressions)	12 s, shared	15 s	82 s	204 s
3D graph (4 expressions)	15 s, shared	18 s	70 s	180 s
Lists, definitions, `init`	—	—	30 s	120 s

- The **watchdog** thread inside the engine checks every 0.2 s. When a deadline has passed, it writes a reply with `timeout:true` and `restart:true` and exits. This is faster than waiting for libqalculate's own abort.
- **Java** (`Engine`) runs one engine process and handles one request at a time. If no reply arrives in time, it kills the process and starts a new one for the next request.

Situation	What Java does	What the page gets
The reply has `restart:true`	Passes the reply on, then stops the process	The engine's reply
No reply in time	Kills the process	Code `timeout`
The user cancels (taps "calculating…")	Kills the process	Code `cancelled`
The engine exits while working on a request	Never resends that request, because it would probably crash again	Code `crash`
Writing the request fails because the engine had already exited	Starts a new engine and sends the request once more	The normal reply, or code `engine`

After a restart, Java restores `ans` with `set_ans`, and libqalculate reloads the user definitions from its files. The user notices nothing, except that `ans2` to `ans5` and temporary variables (such as `x = 5`) are gone.

Since 1.2.0, `set_ans` reads the value with as many digits of precision as it has: a new engine starts at precision 10, and before 1.2.0 a restored approximate `ans` kept only about 38 digits. Numbers of 2,000 digits or more are read directly with GMP and passed in as temporary variables, because libqalculate reads digits one at a time, in time that grows with the square of their number (200,000 digits: 0.2 s instead of 3.4 s on a PC, with exactly the same result).

7. Data, external interfaces and security

All data stays in the app's private storage (basic design 2.7). Every file is first written to a temporary file and then renamed, so a forced stop leaves either the old or the new content.

Data	Where	Notes
History	`state/qalc.history.json`	Up to 300 items. The newest results are kept whole up to 1,000,000 characters in total; older long results keep their first 2,000 characters
Settings	`state/qalc.settings.json`	19 items (lang and theme were added in 1.3.0)
Display language	`state/qalc.lang.json`	"en" or "ja", written only when the user chooses with "Language / 言語"; English when missing (1.3.0)
Expression being typed	`state/qalc.draft.json`	Saved 0.5 s after typing
Graph inputs	`state/qalc.plot.*.json`, `state/qalc.plot3d.*.json`	Expressions, ranges, grid
`ans`	SharedPreferences `engine`, key `ans`	Up to 500,000 characters (an approximate value is saved as interval(lower, upper): about 400,000 characters for 200,000 digits)

Data	Where	Notes
QCalc Pro	SharedPreferences `pro`	owned, price, pre_pro, era, legacy, test_locked (1.3.0). The purchase itself is kept by Google Play on the user's Google account
User-defined variables and functions	`qalculate/definitions/*.xml`	libqalculate's XML format. Unreadable files are moved to `qalculate/broken-definitions/`
Exchange rates	`qalculate/eurofxref-daily.xml`, `btc.json`, `rates.json`	As downloaded
Exported history	Where the user chooses (Android's file picker)	CSV, UTF-8 with a byte order mark (1.3.0). Outside the app's storage, so it stays when the app is removed

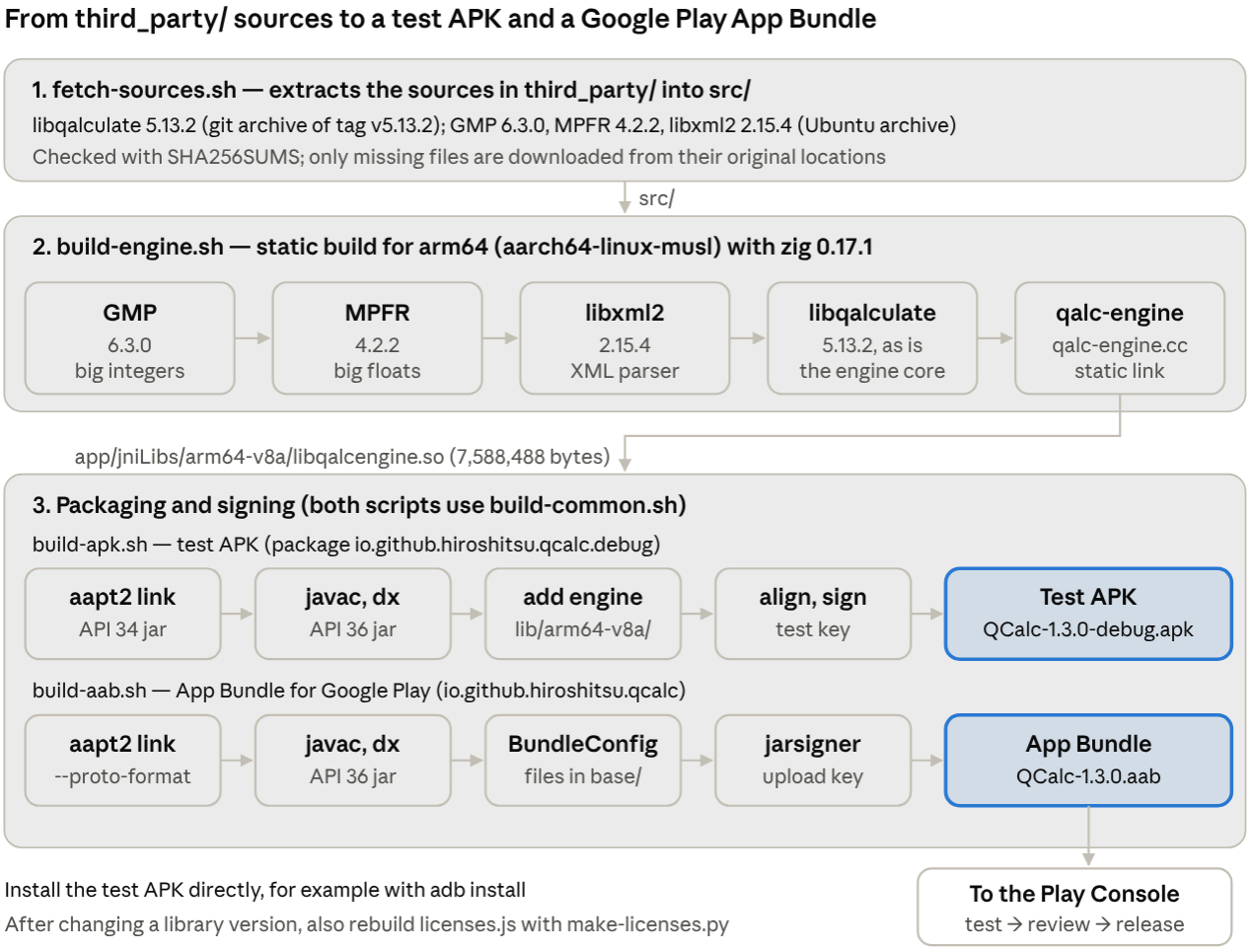
The app's only network traffic is downloading exchange rates over HTTPS from the same three sources that Qalculate! uses (basic design 2.8): the European Central Bank (major currencies), Coinbase (Bitcoin), and currency-api through jsDelivr (other currencies, with a fallback host). The User-Agent is `QCalc/1.3.0 (Android; libqalculate 5.13.2)`. Connections time out after 15 s and reads after 20 s, and replies over 1,000,000 bytes are discarded. In builds with billing, buying and checking QCalc Pro go through the Google Play Store app on the phone (detailed design 3.20).

Security and privacy (basic design 2.10):

- The WebView shows only the bundled pages under `file:///android_asset/`. File and content access are off, and other links open in the browser only if they are http or https.
- Results and user input are inserted with `textContent`, never as HTML.
- The JavaScript bridge checks every request as JSON and rejects requests that contain line breaks. Saved-state names are limited to 64 lowercase letters, digits and `._-`.
- A file that the page asks Java to save must be a CSV file with a plain name, and it is written only where the user picks in Android's file picker (detailed design 3.23).
- libqalculate is built with `--disable-insecure`, which disables `command()`. The engine reads and writes only `files/qalculate/`.

- The app has only two permissions (INTERNET and VIBRATE; builds with billing also get com.android.vending.BILLING from the Billing Library), collects no personal data, and has no ads or analytics. The privacy policy is at https://hiroshitsu.github.io/qcalc/privacy.html.
- The upload key and its password are never published or committed. Google keeps the app signing key.

8. Build and release



Build flow · three stages to the test APK and the App Bundle

Three shell scripts build everything on Ubuntu 24.04 (build and maintenance manual 5.2–5.6). Neither the Android NDK nor Gradle is needed. An Android Studio (Gradle) setup for the same sources is also included; it is also how the build with billing for QCalc Pro is made (qcalc.billing=true, build and maintenance manual 5.11). It has not been tested yet, because the build environment cannot reach Google's Maven repository.

- **Engine**: zig 0.17.1 cross-compiles GMP, MPFR, libxml2 and libqalculate as static libraries (-O2) for `aarch64-linux-musl`, then links `qalc-engine.cc` (C++17) statically with `-Wl,-z,max-page-size=16384`, so that the engine also runs on devices

with 16 KB memory pages. The result is a 7,588,488-byte executable.

- **libqalculate configure options**: `--enable-compiled-definitions` (definitions built into the library), `--disable-insecure` (no `command()`), `--without-libcurl` (Java downloads the rates), `--without-icu`, `--without-gnuplot-call` (the page draws the graphs), `--with-readline=no` and `--disable-nls`.
- **Outputs**: `QCalc-1.3.0-debug.apk` (3,315,796 bytes; a separate package, `io.github.hiroshitsu.qcalc.debug`, signed with a test key) and `QCalc-1.3.0.aab` (3,304,203 bytes, signed with the upload key, SHA-256 71d5f5d83ca313fde21672db30edd82aec0c3554ec0bb2c98b733955056e5b23), both without billing. The bundle is validated with bundletool 1.17.2. Its BundleConfig turns off "uncompressed native libraries", so that the APKs that Google Play makes still extract the engine as a file.
- **Updating libqalculate** (build and maintenance manual 5.9): put the new tagged source in `third_party/`, rebuild, port any upstream changes to the steps copied from `qalc` (`calculate_once`, `store_ans`), check the exchange-rate sources, regenerate the license texts, and rerun all the tests, including the comparison with `qalc`.
- **Releasing an update**: raise `VERSION_CODE` in `version.properties` (5 for 1.3.0), build the bundle, test the debug APK on a phone, and upload the bundle to a Play Console track.

9. Testing

No physical device was available, so each of the four parts was tested on a PC running Ubuntu 24.04 (test specification 6.1):

- **Engine**: the same arm64 engine binary that ships in the APK, run with `qemu-aarch64-static` and compared with the official `qalc` built from the same libqalculate tag.
- **Java**: `Engine.java` and `StateStore.java` run unchanged on OpenJDK 21 with small stand-ins for Android's `Context` and `SharedPreferences`, against both the real engine and a fake one.
- **Page**: the real `app/assets` in Chromium (Playwright) at the size of a phone screen, with requests relayed to the real engine.
- **Packages**: apksigner, zipalign, aapt2, jarsigner and bundletool on the APK and the App Bundle.

Area	Items	Passed
Comparison with the official `qalc`	309	309

Area	Items	Passed
Engine functions, 3D graph values, calculus and matrices, Japanese units	66	66
All built-in examples (on the English and the Japanese screen)	73	73
Java (real engine and fake engine)	24	24
Page: keypads, calculus and matrices, examples, undo, text size, search, history, 3D graphs, fixes	89	89
APK and App Bundle checks	22	22
High precision (1.2.0): 200,000 digits compared digit by digit with mpmath, limits, restoring a long ans	16	16
Java (1.2.0): ans kept with all its digits	7	7
Page (1.2.0): long results shown, copied and saved	32	32
Page (1.3.0): QCalc Pro, color themes, CSV export, the preview, digit grouping	19	19
Page (1.3.0): the language of the screen, and no Japanese left on the English screen	8	8
Total	**665**	**665**

In addition, a build of the engine with AddressSanitizer and UndefinedBehaviorSanitizer ran the functional tests and 33,000 random requests, and the release engine ran 6,000 random requests, with no errors. For 1.3.0, 27 page tests were added (Q01 to Q27), and the comparison with qalc, the engine tests, the examples of both screens, the 31 Java tests, the page tests on the Japanese screen and the 22 package checks were run again with the 1.3.0 build (2026-11-13). The high-precision tests, the long-result page tests and the history-reuse page tests are results of 1.2.0 (test specification 6.1).

Not tested yet (test specification 6.6): installing and running on a real device, TalkBack, a real exchange-rate download, input with a real Japanese keyboard, rendering in the Android System WebView, window insets and predictive back on a device with Android 15 or later, the Android Studio build, a device with 16 KB pages, Google Play's review and closed test, buying QCalc Pro in a build with billing (with a license tester's account), and the language and the CSV export on a phone.

10. Components and versions

All calculation is done by the official libqalculate 5.13.2, whose source is not changed. The app's own code is the bridge program to the engine, the Java code for Android and the page. At every start, the bridge program adds the ten Japanese units and `trace()` to the running engine (libqalculate's files are not changed).

Component	Version and settings	Source	License
App	QCalc 1.3.0 (`io.github.hiroshitsu.qcalc` , versionCode 5), arm64-v8a. App Bundle for Google Play: 3,304,203 bytes	This project (published as HiroshiTsu/qcalc on GitHub)	GPL v2 or later
Calculation engine	libqalculate 5.13.2, with the definitions built in (`--enable-compiled-definitions`) and `command()` and the like disabled (`--disable-insecure`); no libcurl, ICU, gnuplot, readline or NLS	github.com/Qalculate/libqalculate (tag v5.13.2)	GPL v2 or later
Big integers	GMP 6.3.0	The upstream tarball as Ubuntu distributes it (gmp_6.3.0+dfsg)	LGPL v3 or later
Big floats	MPFR 4.2.2	The upstream tarball as Ubuntu distributes it	LGPL v3 or later
XML parser	libxml2 2.15.4	The upstream tarball as Ubuntu distributes it	MIT
Cross compiler	zig 0.17.1 (pip's ziglang), target aarch64-linux-musl, static linking, aligned for 16 KB pages	PyPI	MIT
Android target	minSdk 24 (Android 7.0), targetSdk 36 (Android 16). Java is compiled against android.jar of API 36; aapt2 uses that of API 34	Android SDK Platform 36 and 34 (not in the repository)	—
Signing	Play App Signing (Google keeps the app signing key). The upload key is RSA 4096 bits, certificate SHA-256 73:16:7B:03:1C:34:CC:0D:09:D2:A9:E1:D7:D1:E8:41:34:22:03:20:48:2A:B8:4B:80:0D:DB:0F:95:D2:E3:38. The test APK is signed with a separate test key	The project's upload key (kept in signing/, never published)	—
Android Studio settings	Android Gradle Plugin 8.11.1, Gradle 8.13, compileSdk 36 (building in Android Studio has not been tried)	Google Maven, Gradle	Apache 2.0
C/C++ runtime libraries	musl libc, LLVM libc++, libc++abi, libunwind, compiler_rt (bundled with zig 0.17.1, statically linked)	zig's distribution	MIT, Apache 2.0 with LLVM Exceptions
Billing (builds with billing only)	Google Play Billing Library 8.3.0 (included only when Gradle runs with qcalc.billing=true)	Google Maven	Google's terms (not open source; QCalc's own code carries an exception that allows linking it)

11. Guide to the documents

Document	English	Japanese	What it covers
Overview	Design overview (English) (this tab), 00-overview-en.pdf	概要, 00-overview.pdf	What QCalc is and what changed in 1.3.0, 1.2.0 and 1.1.0; the design in brief; components and versions; glossary; revision history
1. Requirements definition	1. Requirements, 01-requirements-en.pdf	1. 要件定義書, 01-requirements.pdf	Purpose, scope, 22 use cases, 34 functional requirements, non-functional requirements, constraints, 21 acceptance criteria, QCalc Pro
2. Basic design	2. Basic design, 02-basic-design-en.pdf	2. 基本設計書, 02-basic-design.pdf	Components, deployment, modules, screens and transitions, keypads, main flows, data, external interfaces, error handling, security

Document	English	Japanese	What it covers
3. Detailed design	3. Detailed design, 03-detailed-design-en.pdf	3. 詳細設計書, 03-detailed-design.pdf	Class diagram, JavaScript bridge, engine protocol, calculation, engine supervision, page modules, input, history, graphs, exchange rates, data formats, time limits, error codes and messages, Android 13–16, licenses, App Bundle, search, additions to the engine, QCalc Pro, language, color themes, CSV export
4. User manual	4. User manual, 04-user-manual-en.pdf	4. 操作マニュアル, 04-user-manual.pdf	Installation and the language, the screen, calculating (also very long results), calculus, matrices, keyboard input, examples, the menu (graphs, lists, QCalc Pro, exporting the history), settings, troubleshooting, with screenshots

Document	English	Japanese	What it covers
5. Build and maintenance manual	`5. Build and maintenance`, 05-build-and-maintenance-en.pdf	`5. ビルド・保守マニュアル`, 05-build-and-maintenance.pdf	Environment, build flow, sources, engine build, APK, App Bundle, releases, checks, updating libqalculate, maintenance notes, Android Studio and the build with billing
6. Test specification and results	`6. Test specification and results`, 06-test-specification-en.pdf	`6. テスト仕様書・結果`, 06-test-specification.pdf	Test policy and environment, engine tests, Java tests, page tests, package checks (665 items), tests not yet done, known limits

The PDF files are in the design/ folder of the source repository. The steps for publishing QCalc on Google Play (the entries in Play Console, testing and review) are kept in a separate working document that is not part of this set.

12. Glossary

Term	Meaning
libqalculate	The calculation library of the official Qalculate!. It does every calculation in QCalc
Engine	The bridge program `qalc-engine` linked with libqalculate. It ships as `lib/arm64-v8a/libqalcengine.so` and runs as a child process
Preview	The result shown while an expression is being typed
Confirm	Pressing “=”: the expression is calculated with `store:true` and the result is added to the history

Term	Meaning
`ans` event	The line that the engine sends after a confirmed calculation, with the new `ans` (`value`) and the text to insert (`input`)
Watchdog	The thread inside the engine that answers with a timeout and exits when a calculation runs past its limit + 3 s
Message stop	libqalculate's way of holding back warnings and errors during a calculation (`beginTemporaryStopMessages`)
Template	A function call with its arguments laid out, inserted by a key (for example `integrate(, 0, 1)`), with the cursor where the expression goes
Upload key	The key that signs the App Bundle uploaded to Google Play. Google keeps the key that signs the app itself (Play App Signing)
Precision	The number of significant digits used to calculate and show results: 2 to 200,000 in the settings (10 by default). Raised from 1,000 to 200,000 in 1.2.0; since 1.3.0, up to 1,000 without QCalc Pro
QCalc Pro	A one-time product in Google Play (`qcalc_pro`) that unlocks precision above 1,000 digits, 3D graphs, color themes and exporting the history (1.3.0)
Earlier user (legacy)	Someone who used QCalc before Pro began (the phone has a saved history or settings, or ran a build without billing). They keep the Pro features for free, also after updating to a build with billing. Recognized by a mark kept on the phone, which clearing the app's data removes
Display language (`qalc.lang`)	The language of the screen. English is the default; the choice is saved in qalc.lang only when the user picks a language with "Language / 言語" (1.3.0)
Build with billing	A build made with Gradle and qcalc.billing=true. It contains PlayBilling and the Google Play Billing Library 8.3.0, so QCalc Pro can be bought. The shell scripts make builds without billing, which lock nothing

13. Revision history

Date	What	Changes
2026-11-13	Design documents, 5th edition	Revised the overview and the six documents for QCalc 1.3.0, and made English versions of all the documents (seven English tabs, with the same content as the Japanese ones). Added the English and Japanese screens, QCalc Pro, color themes, exporting the history, all digits in the preview, digit grouping and versionCode 5. Redrew the diagrams and added English versions of them. Added the 27 tests of 1.3.0 to the test specification and updated it with the results of 1.3.0
2026-11-13	App QCalc 1.3.0	English and Japanese screens (English by default; “Language / 言語” switches), QCalc Pro (a one-time purchase; the build with billing is made with Gradle), seven color themes, exporting the history (CSV). The preview while typing also shows all the digits (the preview of 4587! had shown only its beginning), and digit grouping puts commas only in the integer part. versionCode 5, together with the fix that makes English the default
2026-11-12	Design documents, 4th edition	Revised the overview, the six documents and the English overview for QCalc 1.2.0: the precision limit (200,000 digits), showing, copying and saving long results, carrying ans over with all its digits, and what happens when copying fails. Added “Very long results” and screenshots to the user manual and retook the screenshots of the settings. Added 55 tests of high precision, Java and the page to the test specification

Date	What	Changes
2026-11-12	App QCalc 1.2.0	Raised the precision limit from 1,000 to 200,000 digits (versionCode 3). Long results are shown and copied with all their digits, and the history keeps the newest ones up to 1,000,000 characters in total. Fixed ans being rounded to about 38 digits after the app closed, and made restoring a long ans fast. The app no longer stops when a result is too long to copy
2026-11-12	Design documents, 3rd edition	Revised the overview and the six documents for QCalc 1.1.0: the key tabs (calculus and matrices), matrices as tables, examples, undo, text sizes, search in Japanese, the Japanese units and trace(), and the workaround for the hidden warnings. Redrew the diagrams (the keys, the screen layout and others), added diagrams such as “Changes in 1.1.0”, and retook the 14 screenshots (27 screens) of the user manual
2026-11-12	App QCalc 1.1.0	Easier to use (versionCode 2): four key pages (123, Functions, Calculus, Matrix), so that derivatives, integrals, limits, differential equations and matrices can be typed with keys; matrix results as tables; the a=b key, examples and tips, undo, text sizes, Japanese names and search, the Japanese units, trace(). Fixed warnings that disappeared after a definite integral with an infinite bound, the text inserted from the unit list, when exact forms are shown, old coordinates in graphs, stopping a calculation and more
2026-11-12	Design documents, 2nd edition	Revised the overview and the six documents for QCalc 1.0.0: 3D graphs, Android 15 and 16, the license screen, the App Bundle and signing, the Google Play materials, review fixes and test results, with new diagrams and screenshots

Date	What	Changes
2026-11-12	App QCalc 1.0.0	The version for Google Play (2026-11-11 to 12): the name QCalc and the package name io.github.hiroshitsu.qcalc, a new icon, targetSdk 36 (screen edges and the back gesture), 3D graphs, the open source license screen, a link to the privacy policy, a fix for the newest history line being hidden, and the App Bundle (Play App Signing). The last review fixed details of the cursor keys, the graph messages and 3D graphs, and a test APK that did not start (the build named the wrong activity to start)
2026-11-09	Source set	Added settings to open the sources in Android Studio (Gradle). Put the resources and the built engine under app/ and the version in version.properties. Checked that the contents of the APK did not change (the first edition's tests)
2026-11-09	Design documents, 1st edition	Wrote the six documents, from the requirements to the tests, for the source code of app 5.13.2-3, with 13 diagrams (use cases, components, deployment, classes, sequences, states, screen transitions and others) and 7 screenshots (13 screens) in the user manual
2026-11-09	App 5.13.2-3	Fixes from three independent code reviews (Android, page, engine): assignments are not run in the preview, one evaluation so that ans equals what is shown, detecting timeouts, supervising and restarting the engine, exact reuse of the history, converting Japanese input, the local currency (yen), saving the state at once and more
2026-11-09	App 5.13.2-2	The version handed out before (signed with the same key, so it can be updated in place)